

Proposal Draft V.1.0.2

Project Name: WeShare

Team Number: 054

Project Member (NetID): Yanran Lu (yanran3) , Zihuang Li (zihuang3), Tianyi Liu (tianyil5), Weicheng Xiao(wx10)

Pitch

[WeShare] is a ridesharing website designed to assist students in school in efficiently finding and sharing rides to their desired destinations. Students in UIUC often find themselves in need of an easy, fast and affordable transportation solution. WeShare allows students to connect, save travel costs, and make commuting more convenient. Whether heading to a flight, home for the holidays, or another city, this platform connects students who want to save their gasoline money and students who want an affordable and easy ride to their desired destination.

Functionality

1. Users can post a ride with details such as departure location, destination, luggage space, and availability.
2. Users can browse and filter available rides based on departure time, location, and luggage space.
3. Users can request to join a ride.
4. Ride posters can approve or decline ride requests.
5. Users can rate drivers based on ride experiences (Rating by Stars).
6. A verification system may be implemented for security.

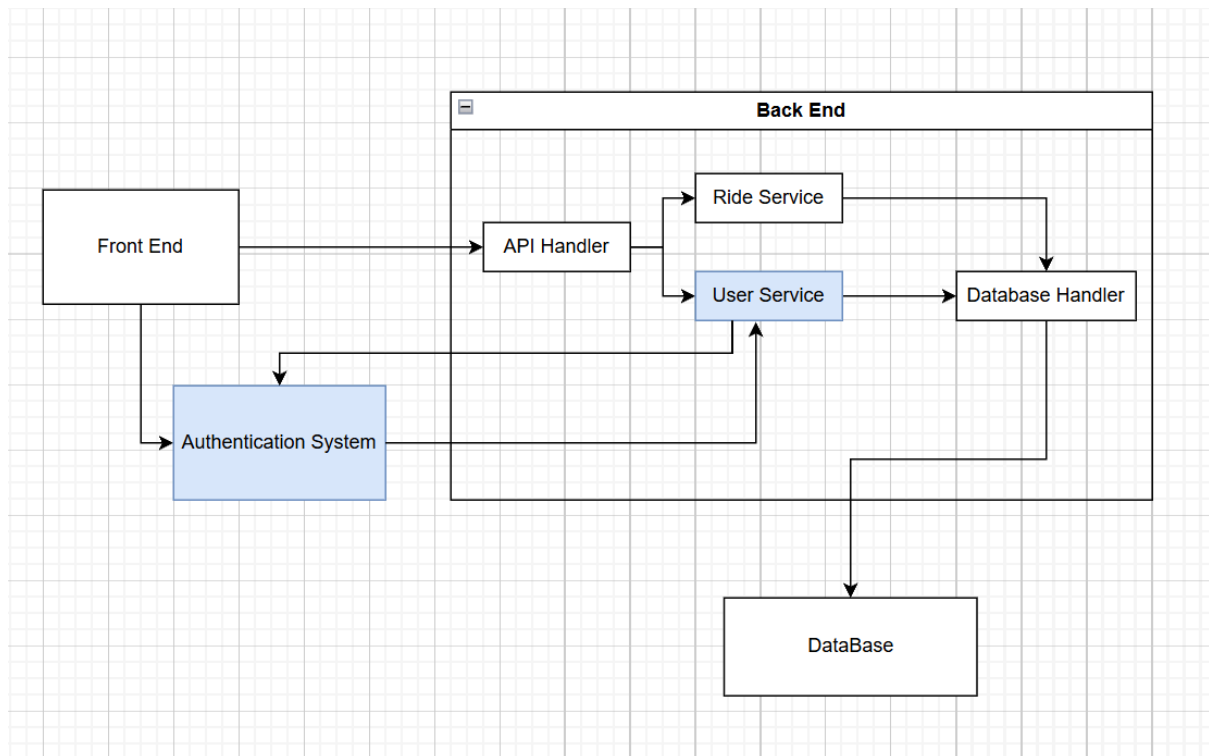
Components

1. **Frontend:**
 - **Functionality:** Displays rides, allows users to post and filter rides, and handles user interactions.
 - **Programming Language:** JavaScript (React) for an interactive and responsive UI.
 - **Major Libraries:** React Router, Axios, etc.
 - **Testing Methodology:** Jest for unit tests and Cypress for end-to-end testing.
 - **Interactions:** Sends and receives data from the backend API.
2. **Backend:**
 - **Functionality:** Stores ride data, handles authentication, and processes ride requests.

- **Programming Language:** Python for robust data handling and scalability.
- **Major Libraries:** SQLAlchemy, WebSocket, etc.
- **Testing Methodology:** Pytest for unit testing, verifying that the API logic is correct. Manual testing for checking edge cases and other expectations. REST API Testing Tool to test API endpoints.
- **Interactions:** Connects with the frontend via REST API, handles database queries.

3. Database:

- **Functionality:** Stores user data, ride details, and transaction history.
- **Database Type:** PostgreSQL for structured and scalable data storage.
- **Testing Methodology:** Database schema validation and SQL query testing.
- **Interactions:** Interfaces with the backend for data retrieval and updates.



PS: Blue tabs are not in MVP, which may be changed in the future.

Continuous Integration

1. **Testing Libraries:** The front-end uses Jest (unit testing) and Cypress (end-to-end testing); the back-end uses Pytest for API testing and Postman for manual testing; the database uses Pytest + SQLAlchemy to check queries and transactions.
2. **Style Guide:** All codes are written in the standard style of their respective languages. If there is no standard, use snake_case. Comments should be provided before important functions or complex methods.
3. **Test Coverage:** We use Jest to ensure unit test coverage on the front end, Cypress for end-to-end testing, and Coverage.py combined with Pytest to calculate test coverage of API logic on the back end to ensure that key functions are fully tested. The minimum coverage is tentatively set at 70%.
4. **Pull Request:** Before merging, team members need to run tests locally, ensure that the code style meets the specifications, and pull the latest code to avoid conflicts.

Schedule

- **Week 1:**
 1. Further analyze project requirements and clarify core functions.
 2. Unify database structure and development environment and complete basic configuration.
- **Week 2:**
 1. Design and build the database and implement the basic table structure.
 2. Build the backend framework and complete the database connection and basic API endpoints.
- **Week 3:**
 1. Initially develop the front-end UI to ensure basic page layout and navigation.
 2. Develop the back-end API to support user registration, login and basic data interaction.
- **Week 4:**
 1. Improve database design, optimize query performance and data storage solutions.
 2. Implement core functions, including user publishing, searching and filtering itineraries.
- **Week 5:**
 1. Test the front-end and back-end interactions to ensure that API calls are correct.
 2. Process user requests to ensure stable and efficient data access.
- **Week 6:**
 1. Initially integrate all modules and conduct system joint debugging tests.
 2. Identify and fix potential functional or logical problems.

- **Week 7:**
 1. Optimize front-end UI/UX to improve interface interaction experience.
 2. Find potential risk issues during the project process.
- **Week 8:**
 1. Strengthen security and improve user authentication and data protection measures.
 2. Conduct internal testing to simulate actual usage scenarios and collect feedback, trying to handle the risks.
- **Week 9:**
 1. Perform complete system testing, including functional, performance, and security testing.
 2. Resolve bugs found during testing and optimize system details.
- **Week 10:**
 1. Complete project reports, PPTs and related documents.
 2. Prepare project presentations and conduct final checks and improvements.

Risks

1. Technical Complexity:

A major challenge we might face is designing and implementing the database for ride data. Ensuring efficient data storage, retrieval, and management while handling a large number of ride postings and user interactions could add significant complexity to the project.

○ **Solution:**

For database design, thorough testing and a robust structure are essential to minimize the risk of failure. Optimizing queries and ensuring efficient data handling will be crucial for maintaining performance and stability.

At the same time, a carefully planned API for user requests is crucial. We would need to ensure that our backend would handle the user's request in an efficient and safe way.

2. Page Scaling Issue

Currently, there may be problems with page scaling because the screen resolutions of web and mobile versions differ. If we aim to maintain a consistent and visually appealing design across all devices, it will require significantly more time and effort.

○ **Solution:**

Optimize the website for responsive design, ensuring that the layout adjusts dynamically based on the user's device. Implement flexible UI elements and test the interface on various screen sizes to maintain usability without excessive development overhead. By doing so, we can balance aesthetics and efficiency, making the platform more user-friendly across different devices.

3. User safety issues

- Since our platform may attract a large number of UIUC students, at the same time, there may also be non-school-related persons (such as external car dealership owners, individual operators, etc.) entering the platform to obtain additional profits, or to cause fraud and malicious behavior.
- Solution: The best solution is to implement an authentication system, such as requiring students to provide their academic ID when registering, and only accepting registrations with school email addresses. A better solution is to connect to UIUC's SSO for more secure identity authentication.

Teamwork

Using a consistent development environment is very important. Since our members have different computer systems, we may create a VM to ensure that the development environments are compatible with each other.

In terms of work allocation, we tentatively decide as follows:

Front End: Yanran Lu (yanran3) + Tianyi Liu (tianyil5)

Yanran is responsible for API communication, Tianyi is responsible for Web construction.

Back End: Zihuang Li (zihuang3) + Weicheng Xiao(wx10)

Zihuang is responsible for database construction, and Weicheng is responsible for core function writing.

In addition, we will discuss and check each other's work and conduct round-trip testing to ensure that everyone is familiar with the whole process.